4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Digital VLSI Design — Assignment Report

Anish Toshniwal (2024102009) Saumya Vira (2024102044)

Repository: https://github.com/anish-006t/4-bit-Array-Multiplier-from-RTL-to-GDSII

1 Introduction

We designed a 4×4-bit unsigned array multiplier with registered inputs and output in Verilog, verified it with an exhaustive self-checking testbench, and took it through a complete open-source RTL-to-GDSII flow on the SkyWater SKY130 130 nm process using the standard cell library `sky130_fd_sc_hd`: logic synthesis with Yosys, static timing analysis with OpenSTA, and place and route with LibreLane (OpenROAD, Magic, KLayout, Netgen).

All tools were run from the IIC-OSIC-TOOLS Docker image (`hpretl/iic-osic-tools:latest`) with the SKY130 PDK selected (`PDK=sky130A`). Tool versions: Icarus Verilog 14.0 (devel), Yosys 0.70, OpenSTA 3.1.0, LibreLane 3.1.0.dev3. The exact command for every step is listed, in order, in the repository's `README.md`.

2 Part A — RTL Design and Verification

2.1 Design

Full adder. `rtl/full_adder.v` implements $sum = a \oplus b \oplus c_{in}$ and $c_{out} = ab + a\,c_{in} + b\,c_{in}$ with two `assign` statements.

Array multiplier. `rtl/mult_array.v` registers `a` and `b` into 4-bit registers `a_r`, `b_r` on every rising clock edge (asynchronous active-low reset `rst_n`). The 16 partial-product bits $pp_i[j] = a_r[j] \cdot b_r[i]$ are formed by 16 AND gates. Row 0 needs no adders and gives $p[0] = pp_0[0]$; rows 1–3 are 4-bit ripple-carry adders of 4 `full_adder` instances each (12 in total, `fa1_0` to `fa3_3`), connected exactly as in the brief's table. The lowest sum bit of each row is a finished product bit; the other sums and the final carry pass down to the next row. The 8-bit result is registered into `p`. The design therefore has $4 + 4 + 8 = 16$ flip-flops and a latency of 2 clock cycles. The `*` operator is not used in `mult_array.v`. Figure 1 shows the structure.

2.2 Testbench

`tb/tb_mult.v` generates a 10 ns clock and holds `rst_n` low for the first 2 cycles (0–20 ns). It applies all 256 combinations ($a, b = 0 \ldots 15$); each pair is applied on a falling clock edge, the testbench waits two rising edges (the 2-cycle latency) and compares `p` with $a \times b$ on the following falling edge using `!==`, so unknown values also count as errors. Every mismatch is printed, followed by exactly one summary line, and the waveform is written to `mult.vcd`.

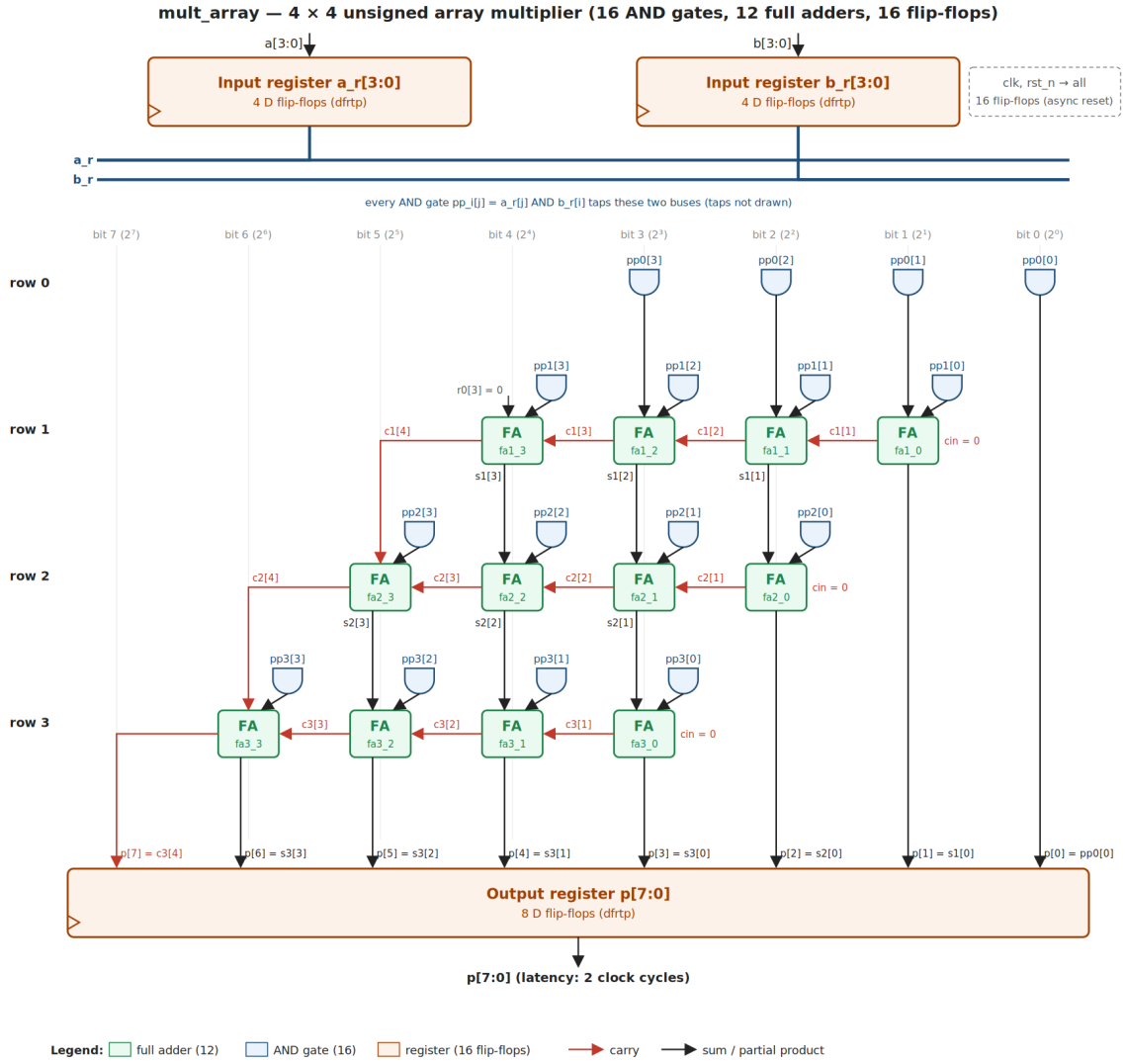


Figure 1: Block diagram of `mult_array`: input registers `a_r`, `b_r` (4 flip-flops each), 16 AND gates producing $pp_i[j]$, 12 full adders in three ripple-carry rows (sums drop to the next row, carries move one bit position left), and the 8-bit output register `p`.

2.3 Results

- RTL simulation: `PASS: 256 vectors checked, 0 errors`
- Error test (`sum` temporarily changed to `a ^ b`): `FAIL: 256 vectors checked, 95 errors`. The first mismatches were `a=3 b=3 expected p=9 got p=5` and `a=3 b=6 expected p=18 got p=10`: the broken adder ignores its carry-in, so every case in which a carry enters an adder fails. After undoing the change the testbench printed PASS again.

2.4 Worked example

We used the roll number of Saumya Vira (2024102044): $A = 4 + 5 = 9$ and $B = 7$, so $A \times B = 63$ (Figure 2). In the testbench this pair is applied at $t_0 = 20 + 20\,(16A + B) = 3040$ ns.

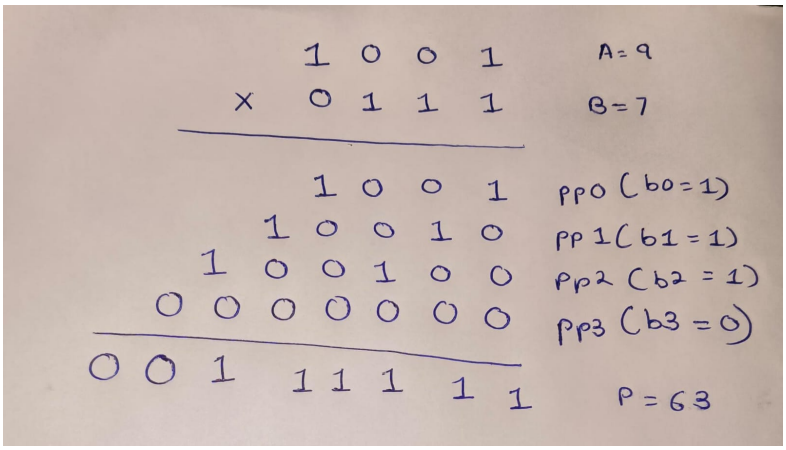


Figure 2: Hand-worked example: $9 \times 7 = 63$ ($1001 \times 0111 = 00111111$), with the four shifted partial products $pp_0 \ldots pp_3$.

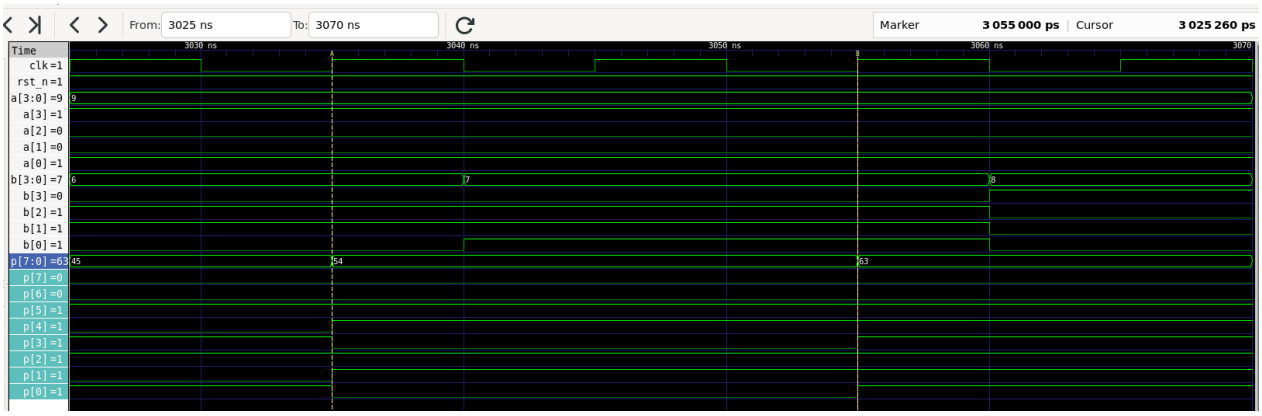


Figure 3: Waveform of 9×7. Marker A: rising edge at 3035 ns (edge 0, the edge before the new inputs). Inputs $a = 9$, $b = 7$ change at 3040 ns, are captured into `a_r`, `b_r` at the rising edge at 3045 ns (edge 1), and $p = 63$ appears at the rising edge at 3055 ns (edge 2, marker B). Latency from A to B: 20 ns = 2 clock cycles. At marker A, p changes to $54 = 9 \times 6$, the product of the previous pair, showing one new result per cycle.

3 Part B — Logic Synthesis with Yosys

`synth/synth.tcl` performs the seven required steps: `read_verilog`, `synth -top mult_array -flatten`, `dfflibmap` and `abc` with the typical-corner Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib`, `opt_clean -purge`, `stat -liberty` (saved to `synth/mult_array_stat.rpt`) and `write_verilog -noattr` (`synth/mult_array_netlist.v`). It was run with `yosys -c synth/synth.tcl`. The gate-level netlist was then simulated with the same testbench, the SKY130 cell models and `-DFUNCTIONAL -DUNIT_DELAY=#1`.

Table 1: Part B results (from `synth/mult_array_stat.rpt`).

Metric	Value
Total number of cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	756.976 μm^2
Flip-flop area	$16 \times 25.024 = 400.384$ μm^2
Flip-flop area (% of total)	52.89 %
3 most-used cell types	`dfrtp_1` × 16, `nand2_1` × 7, `and2_0` × 4 and `maj3_1` × 4 (tie)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The 45 logic cells are: `xor3_1` 3, `xnor3_1` 1, `xor2_1` 2, `xnor2_1` 3, `maj3_1` 4, `nand2_1` 7, `nand3_1` 2, `nand4_1` 2, `nor2_1` 2, `and2_0` 4, `and3_1` 1, `and4_1` 1, `a21oi_1` 3, `a21boi_0` 1, `a22oi_1` 2, `o21bai_1` 1, `o311ai_0` 1, `o2111ai_1` 1, `clkinv_1` 2 and `lpflow_isobufsrc_1` 2.

3.1 Proof that synthesis did not change the behaviour

To check that synthesis preserved the function of the design, the gate-level netlist `synth/mult_array_netlist.v` was simulated with the *same* testbench as in Part A (`tb/tb_mult.v`, unchanged). The netlist contains only the 61 SKY130 standard cell instances produced by Yosys and no RTL code (no `full_adder` module and no `assign` statements describing the array), so the simulation exercises the synthesized circuit itself.

The simulation printed:

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:74: $finish called at 5140000 (1ps)
```

Table 2: RTL simulation versus gate-level simulation with the same testbench.

	RTL (Part A)	Gate-level netlist (Part B)
Design simulated	`rtl/full_adder.v`, `rtl/mult_array.v`	`synth/mult_array_netlist.v` (61 SKY130 cells)
Testbench	`tb/tb_mult.v`	`tb/tb_mult.v` (unchanged)
Vectors checked	256 (all combinations)	256 (all combinations)
Errors	0	0
Result	`PASS`	`PASS`
Simulated time	5140 ns	5140 ns

Because the testbench is exhaustive (all $16 \times 16 = 256$ input pairs), checks every product against $a \times b$ with `!==` (so unknown `x` values would also fail), and waits the full 2-cycle latency, a PASS on the netlist shows that the synthesized circuit produces the correct product for *every* possible input, with the same 2-cycle latency and the same reset behaviour as the RTL. Synthesis therefore did not change the behaviour of the design.

4 Part C — Static Timing Analysis with OpenSTA

`sta/sta.tcl` reads the typical-corner Liberty file and the Part B netlist, creates a clock of period T on `clk`, sets 1.0 ns input delay (all inputs except `clk`) and 1.0 ns output delay, reports the worst setup slack and writes the critical path to `sta/mult_array_timing.rpt`. It was run with `sta sta/sta.tcl`, first with $T = 10$ ns and then with $T = T_{min}$.

At $T = 10$ ns the worst setup slack is WNS $= 7.213$ ns, so $T_{min} = 10 - 7.213 = 2.787$ ns and $F_{max} = 1000/2.787 = 358.8$ MHz. Rerunning with $T = 2.79$ ns gives a slack of $+0.003$ ns ($2.79 - 2.787$, the rounding). The critical path is the same in both runs.

Table 3: Part C results (typical corner, `tt_025C_1v80`).

Metric	Value
Worst slack at $T = 10$ ns	7.213 ns
T_{min}	$10 - 7.213 = 2.787$ ns
F_{max}	$1000/2.787 = 358.8$ MHz
Slack at $T = T_{min}$ (2.79 ns)	+0.003 ns
Critical path start point	input register `a_r[1]` (`dfrtp_1`, instance `_91_`)
Critical path end point	output register `p[6]` (`dfrtp_1`, instance `_88_`)
Number of cells on the critical path	10 (2 flip-flops + 8 logic cells)
Latency $= 2 \times T_{min}$	$2 \times 2.787 = 5.574$ ns
Throughput	358.8 million results per second

Table 4: Cells on the critical path, in order (from `sta/mult_array_timing.rpt`).

#	Instance	Cell	Delay (ns)	Time (ns)	Function
1	`_91_`	`dfrtp_1`	0.398	0.398	launch flip-flop `a_r[1]` (clock-to-Q)
2	`_37_`	`clkinv_1`	0.073	0.471	inverts `a_r[1]`
3	`_42_`	`o311ai_0`	0.356	0.827	AND gates merged with first row-1 carry
4	`_45_`	`a21oi_1`	0.153	0.980	early carry logic
5	`_51_`	`maj3_1`	0.402	1.382	full-adder carry
6	`_58_`	`xnor3_1`	0.370	1.752	full-adder sum
7	`_61_`	`maj3_1`	0.368	2.119	full-adder carry
8	`_63_`	`maj3_1`	0.354	2.474	full-adder carry
9	`_64_`	`xor3_1`	0.178	2.652	full-adder sum (`fa3_3.sum`)
10	`_88_`	`dfrtp_1`	—	—	capture flip-flop `p[6]` (setup 0.135 ns)

Full-adder cells on the critical path. Of the 8 logic cells, 5 come from full adders (3 `maj3_1` carries and the `xnor3_1`/`xor3_1` sums); the other 3 (`clkinv_1`, `o311ai_0`, `a21oi_1`) are merged AND-gate and carry logic. This is fewer than the estimate of about $2N = 8$ full-adder delays. This is because the first adder of each row is really a half adder ($c_{in} = 0$), which synthesis replaced with cheaper logic, and the partial-product AND gates were merged into those cells, so they add no separate delay.

5 Part D — Physical Design: RTL to GDSII

`pnr/config.json` sets `DESIGN_NAME = "mult_array"`, the two Verilog files, `CLOCK_PORT = "clk"`, `FP_SIZING = "absolute"` and `DIE_AREA = [0, 0, 150, 150]`. The starting clock period was $1.2 \times T_{min} = 1.2 \times 2.787 = 3.344$ ns, rounded up to 3.5 ns. The flow was run with `librelane config.json` from `pnr/`; whenever timing failed, `CLOCK_PERIOD` was increased by 0.5 ns (Table 5).

Table 5: All LibreLane runs (setup slack = worst over all 9 signoff corners).

Run	`CLOCK_PERIOD` (ns)	Setup slack (ns)	Typical-corner slack (ns)	DRC	LVS
1	3.5	−2.546	+0.408	0	0
2	4.0	−2.027	+0.929	0	0
3	4.5	−1.527	+1.429	0	0
4	5.0	−1.027	+1.929	0	0
5	5.5	−0.527	+2.429	0	0
6	6.0	−0.037	+2.911	0	0
7	6.5	−0.205	+3.111	0	0
8	7.0	−0.116	+3.526	0	0
9	7.5	−0.138	+3.722	0	0
10	**8.0**	**+0.362**	+4.222	**0**	**0**

DRC and LVS were clean in every run, and the typical corner always met timing. All setup violations were in the slow corners (`ss_100C_1v60`: slow transistors, 100 °C, 1.6 V), where each cell is about 2.5 times slower (e.g. `maj3_1` ≈ 0.92 ns instead of 0.37 ns), whereas Part C analysed only the typical corner. Up to 6.0 ns each 0.5 ns step improved the slack by 0.5 ns. From 6.5 ns the slack stayed slightly negative, because LibreLane re-synthesizes and repairs the design for every new period: with a relaxed target it chooses smaller, slower logic. At 8.0 ns the flow completed with all checks clean.

Table 6: Part D results (final run, `pnr/metrics.json`).

Metric	Value
`CLOCK_PERIOD` used	8.0 ns
Number of runs needed	10
Core area	17 759.5 μm^2 (die: $150 \times 150 = 22\,500\,\mu$m^2)
Cells after synthesis (Part B)	61
Cells after place and route	1884
Kinds of cells added	fill, decap, tap, buffers (timing repair), clock buffers (Table 7)
Wirelength	2518 μm
Setup slack	+0.362 ns (worst corner)
Post-route F_{max}	$1000/(8.0 - 0.362) = 130.9$ MHz
F_{max} from Part C	358.8 MHz
DRC errors	0 (Magic 0, KLayout 0, router 0)
LVS errors	0

Table 7: Composition of the 1884 cells after place and route.

Kind	Count	Cells
Logic (flip-flops, gates, inverters)	62	`dfrtp_1` × 16, 42 multi-input gates, 4 inverters
Tap cells	245	`tapvpwrvgnd_1`
Fill cells	254	`fill_1` × 213, `fill_2` × 41
Decap cells	1299	`decap_3`, `decap_4`, `decap_6`, `decap_8`, `decap_12`
Timing-repair buffers	21	`buf_2` × 17, `clkdlybuf4s25_1` × 2, `clkdlybuf4s50_1` × 2
Clock-tree buffers	3	`clkbuf_16`
Total	1884	

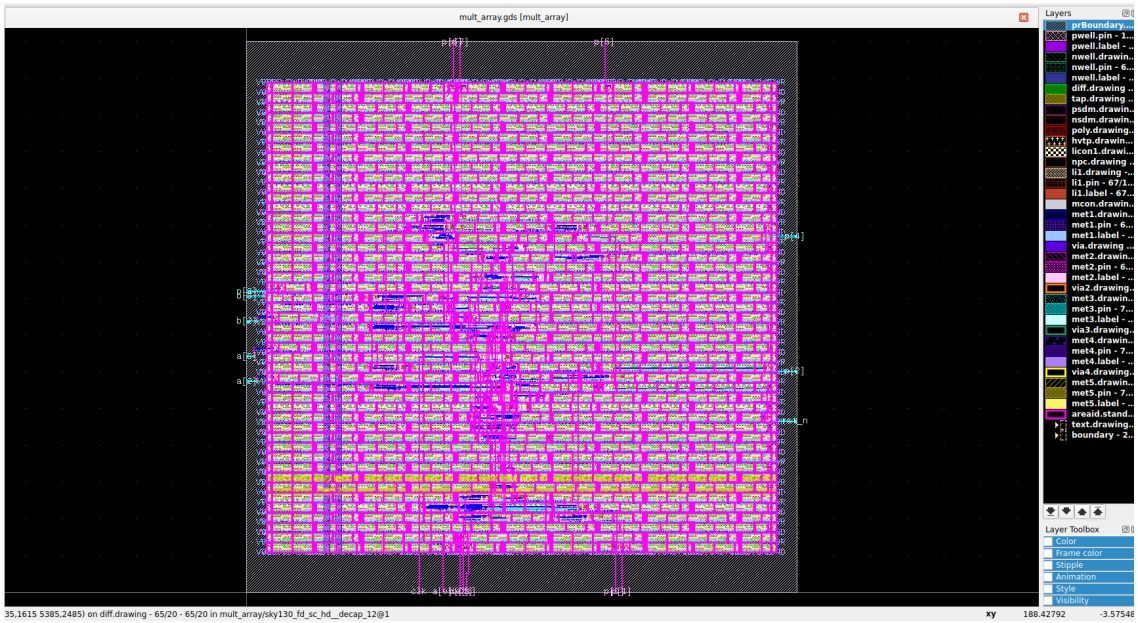


Figure 4: Final layout in KLayout: the whole $150 \times 150\,\mu$m die with the I/O pins on the edges, the standard cell rows and the routing.



Figure 5: Zoomed view of the logic region: standard cell rows bounded by the VPWR/VGND rails, with signal routing on met1 (blue, horizontal) and met2 (magenta, vertical). Decap filler cells fill the rows on both sides.

6 Answers to the Questions

1. Logic cells after synthesis. Synthesis produced $61 - 16 = 45$ logic cells, not $16 + 12 = 28$. The 16 AND gates and 12 full adders are the structure we wrote, not the cells the chip uses: `-flatten` removed the module boundaries and ABC re-mapped the logic onto the cells that suit it best. A full adder is not one cell here: ABC cannot map onto the two-output `fa_1` cell, so each adder becomes a sum cell (`xor3`, `xnor3` or two-input XOR/XNOR gates) and a carry cell (`maj3` or NAND/AOI logic), which raises the count. Some logic disappears, because the first adder of each row has $c_{in} = 0$ (a half adder) and $r_0[3] = 0$, and most AND gates are merged into larger cells

such as `a21oi`, `a22oi`, `nand3`/`nand4`, `o2111ai` and `lpflow_isobufsrc` (function $A \cdot \overline{SLEEP}$); only 4 plain `and2` cells remain.

2. Critical path. The critical path starts at the input register bit `a_r[1]` and ends at the output register bit `p[6]`, whose input is the sum of the last full adder `fa3_3`. It contains 10 cells: the launch and capture flip-flops and 8 logic cells (Table 4), with a data arrival time of 2.652 ns. Of the 8 logic cells, 5 are full-adder functions (3 `maj3` carries, 2 `xor3`/`xnor3` sums) and 3 are partial-product AND gates merged with the first carries of row 1. This is fewer than the estimate of about $2N = 8$ full-adder delays, because synthesis simplified the half adders at the start of each row into cheaper cells, merged the AND gates into them, and rebalanced the logic. The shape of the path still matches the theory: it ripples through the carry chain across the rows and ends at the last adder's sum feeding a high product bit.

3. Flip-flop area. The 16 flip-flops take 400.384/756.976 = 52.89 % of the total cell area. One `dfrtp_1` is 25.024 μm^2 (two latches plus reset logic), whereas a `nand2_1` is 3.75 μm^2 and most gates are 4–9 μm^2. The design has 16 flip-flops but only 45 small logic cells, so the registers dominate. For an N-bit multiplier the logic grows as N^2 while the registers grow only as $4N$, so the flip-flop share falls for larger designs.

4. Post-route F_{max} and added cells. Post-route $F_{max} = 1000/(8.0 - 0.362) = 130.9$ MHz, much lower than 358.8 MHz from Part C, for four reasons. (i) Signoff uses the worst corner (slow transistors, 100 °C, 1.6 V), whereas Part C used the typical corner; with the post-route typical-corner slack (+4.222 ns) the result is $1000/(8.0 - 4.222) = 264.7$ MHz. (ii) Real wires add resistance and capacitance (extracted after routing), so cells drive larger loads and switch more slowly; Part C had no wires. (iii) Part C assumed an ideal clock; after CTS the clock passes through buffers, the flip-flops see slightly different arrival times (skew), and LibreLane subtracts a 0.25 ns clock uncertainty. (iv) LibreLane re-synthesized the logic for the 8.0 ns target and inserted repair buffers, so the critical path is longer than in Part C. The flow added: *tap cells* (245) that tie the n-well and substrate to VPWR/VGND to prevent latch-up; *fill cells* (254) that keep wells and power rails continuous along each row, as the design rules require; *decap cells* (1299), decoupling capacitors that stabilise the supply when many cells switch at once; *timing-repair buffers* (21) that split long or heavily loaded nets to fix slew/capacitance limits and timing; and *clock buffers* (3 `clkbuf_16`) that distribute the clock to the 16 flip-flops with low skew.

5. DRC and LVS. DRC (Design Rule Check) verifies the geometry of the layout against the foundry's manufacturing rules: minimum widths and spacings, via enclosures, minimum areas, well and density rules. LVS (Layout Versus Schematic) extracts the circuit actually drawn in the layout (devices and connections) and compares it with the intended netlist, finding opens, shorts, missing or wrong devices and pins. Both must pass because they check different things: DRC ensures the chip can be manufactured reliably, LVS ensures the manufactured chip is the circuit we designed. A layout can satisfy one and fail the other, and fabrication is too expensive and slow to discover such errors afterwards. Our final layout has 0 DRC errors (Magic and KLayout) and 0 LVS errors.

6. 8-bit multiplier estimate. From Section 2 of the brief, an N-bit array multiplier needs N^2 AND gates and $N(N-1)$ full adders, and its critical path is about $2N$ full-adder delays.

- AND gates: $8^2 = 64$ instead of $4^2 = 16$ (4×).
- Full adders: $8 \times 7 = 56$ instead of $4 \times 3 = 12$ (4.7×); hardware grows quadratically.
- Critical path: $2 \times 8 = 16$ full-adder delays instead of $2 \times 4 = 8$ (2×); delay grows linearly.
- Delay estimate from our Part C result: the critical path's logic delay is $2.652 - 0.398 = 2.254$ ns for about 8 full-adder delays, i.e. ≈ 0.282 ns per full-adder delay. For 8 bits: $16 \times 0.282 = 4.51$ ns of logic, so $T_{min,8} \approx 0.398 + 4.508 + 0.135 = 5.04$ ns and $F_{max,8} \approx 1000/5.04 \approx 198$ MHz (versus 2.787 ns and 358.8 MHz for 4 bits; the same formula reproduces the 4-bit value $0.398 + 2.254 + 0.135 = 2.787$ ns).

- Registers: $4N = 32$ flip-flops instead of 16.

Doubling the width therefore roughly quadruples the adder hardware but only doubles the critical-path delay.

7 Team Contributions

Table 8: Contributions by part.

Part	Done by	Work
A: RTL design and verification	Both	**Anish:** `full_adder.v`, `mult_array.v` (partial products, adder-row connections, registers and reset), block diagram. **Saumya:** `tb_mult.v` (exhaustive self-checking testbench), error test, hand-worked example, waveform with latency marking.
B: Logic synthesis	Anish Toshniwal	`synth.tcl`, synthesis run, Part B table, gate-level netlist simulation, Questions 1 and 3.
C: Static timing analysis	Saumya Vira	`sta.tcl`, runs at $T = 10\,\text{ns}$ and $T = T_{min}$, Part C table, critical-path cell list, full-adder comparison, Question 2.
D: Physical design	Both	**Anish:** `config.json`, LibreLane runs and timing closure (10 runs), DRC/LVS checks, Question 5. **Saumya:** Part D table from `metrics.json`, post-route F_{max} comparison, analysis of added cells, layout screenshots, Question 4.
Report	Both	**Anish:** Parts A, B and D sections, README. **Saumya:** Introduction, Part C section, Question 6, conclusion. Both reviewed the final report.

8 Conclusion

The 4-bit array multiplier passed all 256 test vectors in RTL and gate-level simulation, and the testbench detected an injected full-adder fault (95 errors). Synthesis produced 61 cells ($756.976\,\mu\text{m}^2$, 52.89 % flip-flop area). Pre-layout STA in the typical corner gave $T_{min} = 2.787\,\text{ns}$ ($F_{max} = 358.8\,\text{MHz}$) with the critical path from `a_r[1]` to `p[6]`. The LibreLane flow produced a DRC- and LVS-clean GDSII layout on a $150 \times 150\,\mu\text{m}$ die at an 8.0 ns clock (setup slack +0.362 ns, post-route $F_{max} = 130.9\,\text{MHz}$). The gap between the two frequencies is mostly due to the slow signoff corner, followed by wire parasitics, the real clock tree and re-synthesis.

9 Use of AI Tools

We used an AI assistant (Claude) to support parts of this work: setting up the Docker environment, suggesting drafts of the Verilog code and tool scripts, helping to interpret the tool reports and the LibreLane timing failures, and helping to structure this report.

References

[1] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Pearson, ch. 11.

[2] J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Pearson.

[3] "IIC-OSIC-TOOLS." [Online]. Available: https://github.com/iic-jku/IIC-OSIC-TOOLS

[4] “SkyWater SKY130 PDK.” [Online]. Available: https://skywater-pdk.readthedocs.io
[5] “Yosys Open SYnthesis Suite.” [Online]. Available: https://yosyshq.readthedocs.io/projects/yosys
[6] “OpenSTA.” [Online]. Available: https://github.com/parallaxsw/OpenSTA
[7] “LibreLane.” [Online]. Available: https://github.com/librelane/librelane
[8] “OpenROAD.” [Online]. Available: https://openroad.readthedocs.io
[9] “KLayout.” [Online]. Available: https://www.klayout.de